

PyQUDA 仓库分析报告：

QUDA 的 Python 接口库 (Cython + CuPy + MPI)

opencode analy

2026-08-17

摘要

本报告对 `/root/PyQCD/refer/git-rep/PyQUDA/` (PyQUDA: 以 Cython 编写的 QUDA GPU 格点 QCD 库 Python 封装) 进行全库只读分析。覆盖 346 个文件 (py 146 / h 137 / 配置与文档 63), 以三视角 (主体结构、各部分关系、项目思路) 掌握项目全貌, 并以 “**PyQUDA 传播子反演全流程**” 为主题, 按 A 代码工作 15 步框架逐项重现。核心结论: PyQUDA 以 `pyquda_core/pyquda/src/quda.in.pyx↔quda.h` 的 Cython 绑定为地基, `pyquda_comm`(MPI 网格 + 张量后端 + 格点场数据类) 为枢纽, `pyquda_utils/pyquda_io/pyquda_plugins` 为工具与插件, `examples/tests` 为工作流与验证; 整体呈现 “`utils/io/plugins → core → QUDA C++ API`” 的依赖主链。本库**非独立 git 仓库** (被 PyQCD 主仓库跟踪), git 信息反映主仓库历史。

目录

1 仓库概览

1.1 目录结构

被分析库根目录为 `refer/git-rep/PyQUDA/`。目录树 (实测, 排除 `.git` 与 `__pycache__`) 如下:

```
1 PyQUDA/
2 |-- pyquda_core/          # 核心包 (独立构建, pip 包名 PyQUDA)
3 |   |-- pyquda/           # 主包: field/dirac/action/hmc/gamma/quda.pyi
4 |   |   |-- src/          # Cython 绑定源: quda.in.pyx / quda.in.pxd
5 |   |-- pyquda_comm/      # 公共层: MPI 网格 / 张量后端 / 格点场数据类
6 |   |   |-- src/          # pointer.pyx (ndarray 指针桥接)
7 |   |-- quda/             # QUDA 头文件 (enum_quda.h/quda.h 等, vendored)
8 |   |-- pycparser/        # 第三方捆绑子模块 (C 头文件解析器, 仅构建期用)
9 |   |-- setup.py          # 构建 pyquda.quda/malloc_quda/pointer 扩展
10 |   |-- pyquda_pyx.py     # 由 quda.h 自动生成 pxd/pyx 的代码生成器
11 |-- pyquda_io/           # IO 包: Chroma/ILDG/MILC/KYU/XQCD/NERSC/OpenQCD/NPY
12 |-- pyquda_plugins/      # 插件框架: pycontract / pygwu / 自动代码生成
13 |-- pyquda_utils/        # 工具包: core/source/gamma/fft/hmc_param 等
14 |-- examples/           # 端到端示例 (HMC/传播子/2pt/PCAC/色散)
15 |-- tests/              # 测试 (weak_field 弱场 + Chroma 参考 + 各功能)
16 |-- docs/               # 报告产物 (本报告与前序 pure 报告)
17 |-- setup.py / pyproject.toml / README.md / .github/ # 打包与 CI
```

1.2 规模统计与 git 信息

指标	实测值
文件总数	346
Python 文件数	146
C/C++ 头文件数	137 (含 pycparser 捆绑的 fake_libc 头)
文档文件数	4 (README.md ×2 + examples/README.md + 本库)
Python 总行数 (core+io+plugins+utils+examples+tests)	31 829
提交数 (主仓库视角)	— (非独立仓库, git log 反映主仓库)
标签数 (主仓库视角)	bug0/dev0-3 等 (非本库标签)

表 1: 仓库实测统计 (数据来源: find/wc/git 实测)

git 说明: 本库不是独立 git 仓库——它位于 PyQCD 主仓库的 refer/git-rep/ 下, gitlog/gittag 反映的是主仓库历史 (HEAD 为 e953b47, 主分支 main)。本报告所有 git 相关信息一律注明“非独立仓库”, 不据此推断 PyQUDA 自身版本演进。

参考背景

参考背景: PyQUDA 官方仓库为 <https://github.com/CLQCD/PyQUDA.git>, README 自述“Python wrapper for QUDA written in Cython” (README.md:3), 基于 QUDA develop 分支, 兼容 <https://github.com/lattice/quda/pull/1489> 之后的任意 commit (README.md:7)。本报告以当前 vendored 代码为唯一实证来源。

上一节给出了仓库的目录骨架与规模, 接下来回答“仓库如何分层组织”——即**主体结构**。

2 主体结构

PyQUDA 按“入口—核心—公共层—工具—插件—示例—测试—构建”分层组织。核心分层如下:

层次	目录	职责
入口层	pyquda_core/pyquda/__init__.py	进程初始化总入口: init() 依次完成 MPI 网格、设备、QUDA 初始化 (pyquda_core/pyquda/__init__.py:99-188)
入口层	pyquda_core/pyquda/__main__.py	CLI 入口 python -m pyquda: 解析 grid/latt/backend 参数后执行用户脚本 (pyquda_core/pyquda/__main__.py:6-74)
公共层	pyquda_core/pyquda_comm/__init__.py	MPI 通信世界、GPU 网格划分 (getDefaultGrid 按通信量最小化选型)、MPI-IO 读写、Logger (pyquda_core/pyquda_comm/__init__.py:135-156,222-262)

层次	目录	职责
公共层	pyquda_core/pyquda_comm/field.py	格点场数据类型: LatticeInfo / LatticeFermion / LatticeGauge / LatticePropagator 等, 含奇偶 (evenodd) 分块与坐标映射 (pyquda_core/pyquda_comm/field.py:101-184,1209-1252)
公共层	pyquda_core/pyquda_comm/array.py	张量后端抽象: numpy/cupy/dpnp/torch 四后端统一接口 (pyquda_core/pyquda_comm/array.py:6-7,19-78)
核心层	pyquda_core/pyquda/dirac/	Dirac 算符家族: FermionDirac/StaggeredFermionDirac 抽象基类 + WilsonDirac/CloverWilsonDirac/StaggeredDirac 具体实现 + general.py 参数工厂 (pyquda_core/pyquda/dirac/abstract.py:150-405,general.py:150-405)
核心层	pyquda_core/pyquda/action/	HMC 作用量: GaugeAction / CloverWilsonAction / HISQAction 与有理逼近参数 (pyquda_core/pyquda/action/abstract.py:19-34)
核心层	pyquda_core/pyquda/src/	Cython 绑定: quda.in.pyx (约 70 个 QUDA 函数直绑) + quda.in.pxd (pyquda_core/pyquda/src/quda.in.pyx:187-476,quda.in.pxd:187-476)
核心层	pyquda_core/pyquda/gamma.py	γ 矩阵代数: DeGrand-Rossi/Dirac-Pauli 基、稀疏乘法、Projector (pyquda_core/pyquda/gamma.py:28-73,215-313)
工具层	pyquda_utils/	core.py (invert 编排/gatherLattice/getDirac)、source.py (源构造)、fft.py (MPI 转置 FFT)、hmc_param.py (参数库) 等 (pyquda_utils/core.py:61-82,315-385,source.py:229-261)
工具层	pyquda_io/	多格式 IO: Chroma(QIO)/ILDG/MILC/KYU/XQCD/NERSC 带 SciDAC 校验和 (pyquda_io/__init__.py:1-55,chroma.py:10-68,io_utils.py:10-68)
插件层	pyquda_plugins/	pycontract (收缩) 与 pygw (overlap 费米子) 插件, 及 pycparser 自动代码生成器 (pyquda_plugins/plugin_pyx.py:232-281)
示例层	examples/	端到端工作流: HMC、夸克传播子、 π/p 2pt、PCAC、色散 (examples/2_Quark_Propagator.py:1-39)

层次	目录	职责
测试层	tests/	以 weak_field.lime 弱场 + Chroma 参考输出做逐位验证 (tests/test_clover.py:1-21, check_pyquda.py:1-31)
构建层	pyquda_core/setup.py	编译 pyquda.quda / pyquda.malloc_quda / pyquda_comm.pointer 扩展, 链接 libquda (pyquda_core/setup.py:8-55)

表 2: 主体结构分层 (数据来源: 全库索引实测)

要点

要点: 本库的分层逻辑是“公共层承载一切跨进程/跨后端能力, 核心层只做 QUDA 薄封装”。pyquda_comm(MPI+ 后端 + 数据类)被 pyquda、pyquda_io、pyquda_plugins、pyquda_utils 四方共用, 是整个依赖图的最底层 Python 面; 真正调用 QUDA 的只有 pyquda/dirac/action/hmc 三层。

对象映射: 本库代码是典型的“代码符号 $\leftrightarrow$ 格点 QCD 物理对象”对应, 主体结构节先给出映射表, 主题分析节将逐符号展开。

代码符号	对象概念	物理/工程含义与参考源
LatticeInfo	格点几何信息	晶格尺寸/奇偶/边界/各向异性; pyquda_core/pyquda_comm/field.py:101-115
LatticeGauge	规范场 $U_\mu(x)$	4+1 维 link 场, 单位元初始化; pyquda_core/pyquda_comm/field.py:1177-1188
LatticeFermion	夸克场 $\psi(x)$	奇偶分块自旋-色旋量; pyquda_core/pyquda_comm/field.py:1209-1212
LatticePropagator	夸克传播子	$S(x, y) = \psi(y)\bar{\psi}(x)$ 的 $4 \times 4 \times 3 \times 3$ 分量组装 (setFermion); pyquda_core/pyquda_comm/field.py:1219-1224
evenodd/lexico	奇偶格点重排	棋盘格 checkerboard 分块 $\leftrightarrow$ 字典序; pyquda_core/pyquda_comm/field.py:117-161
invert	线性求解	解 $(D + m)\psi = \eta$ 得传播子; pyquda_core/pyquda/dirac/abstract.py:212-216
kappa	跳跃参数	$\kappa = (2(m + 1 + \frac{N_d-1}{\xi}))^{-1}$; pyquda_core/pyquda/dirac/wilson.py:19
wall/point	源构造	墙源/点源注入右端项 η ; pyquda_utils/source.py:25-66
stoutSmear	链接平滑	stout 迭代改进信噪比; pyquda_core/pyquda/field.py:224-235

代码符号	对象概念	物理/工程含义与参考源
gatherLattice	MPI 归并	子格点数据聚到根进程还原整格点; pyquda_utils/core.py: 315-385

表 3: 对象映射表 (代码符号 $\leftrightarrow$ 物理/工程对象, 数据来源: 源码直读)

主体结构回答了“仓库是什么”。接下来回答“各部分之间如何连接”——即**各部分关系**。

3 各部分关系

本库的模块依赖呈严格的**单向分层**: 工具/IO/插件只向下依赖核心与公共层, 核心层只依赖公共层与 QUDA C++ API, 公共层不依赖任何上层。

要点

依赖主链: `pyquda_utils / pyquda_io / pyquda_plugins → pyquda → pyquda_comm → Cython 绑定 (quda.in.pyx) → QUDA C++ API(quda.h)`

依赖关系逐条证据如下 (每条均以 `import` 语句核实):

- `pyquda_utils → pyquda / pyquda_comm`: `pyquda_utils/core.py:7-19` 从 `pyquda` 引入 `init/getDirac` 等, `pyquda_utils/core.py:21-52` 从 `pyquda.field` 引入全部格点场类型;
- `pyquda_io → pyquda_comm`: `pyquda_io/mpi_utils.py:3-15` 复用 `pyquda_comm` 的 MPI-IO 与网格函数 (IO 不直接碰 QUDA);
- `pyquda_plugins → pyquda_utils`: `pyquda_plugins/pycontract/__init__.py:5-9` 引入 `pyquda_utils.gamma.Gamma` 与 `pyquda_comm.field`;
- `pyquda → pyquda_comm`: `pyquda_core/pyquda/field.py:8-31` 全量 re-export `pyquda_comm.field`, 并添加 `smear/flow/gauge fixing` 等高层方法;
- `pyquda/dirac → src/quda.in.pyx → quda.h`: `pyquda_core/pyquda/dirac/abstract.py:14-27` 从 `..quda` 引入 `invertQuda/MatQuda/dslashQuda`; Cython 层 `pyquda_core/pyquda/src/quda.in.pyx:13-14` 以 `cimport quda` 直连 QUDA 头文件;
- **构建期生成链**: `pyquda_core/pyquda_pyx.py:140-448` 用 `pyparser` 解析 `quda.h` 自动生成 `enum_quda.py / quda.pxd / quda.pyx`; `pyquda_core/setup.py:8-10` 在构建时调用该生成器;
- **插件生成链**: `pyquda_plugins/plugin_pyx.py:232-281` 同法为任意 C 头生成插件 `pyx/pyi`; `pyquda_plugins/__main__.py:73-99` 以 `python -m pyquda_plugins -l lib -i header` 驱动。

共享枢纽: `pyquda_comm` 是四方共用的枢纽层, 其四块能力——(1) MPI 网格 (`pyquda_core/pyquda_comm/__init__.py:265-373`); (2) 张量后端 (`pyquda_core/pyquda_comm/array.py:19-78`); (3) 指针桥接 (`pyquda_core/pyquda_comm/src/pointer.pyx:58-71`); (4) 格点场数据类 (`pyquda_core/pyquda_comm/field.py`) ——构成“进程 → 设备 → 内存 → 物理场”的完整抽象。

各层协作形成一个典型调用:`examples → pyquda_utils.core.invert → pyquda.dirac.WilsonDirac.invert → (src/quda.in.pyx)invertQuda → QUDA`。第 5 节将沿此链完整走一遍。

关系节回答了“各部分怎么连”。接下来回答“为什么这样设计”——即**项目思路**。

4 项目思路

4.1 设计意图

- **用 Python 驱动 GPU 格点 QCD**: README 开宗明义“Python wrapper for QUDA written in Cython”, “benefit from the optimized linear algebra library CuPy in Python based on CUDA” (README.md:3-5) ——目的不是重实现, 而是把 QUDA 的求解器/Dslash/多重网格以零拷贝方式暴露给 Python 生态;
- **张量后端可插拔**: `pyquda_core/pyquda_comm/array.py:6-7` 声明 `numpy/cupy/dpnp/torch` 四后端与 `cpu/cuda/hip/sycl` 目标, `backendDeviceAPI` 统一设备枚举与切换(`pyquda_core/pyquda_comm/array.py`) ——面向 NVIDIA (CUDA) /AMD (HIP) /Intel (SYCL) 全平台;
- **MPI 多卡一体**: `initGrid` 内置 `getDefaultGrid` 通信量最小化网格划分(`pyquda_core/pyquda_comm/__init__.py:376-426`) 按 `hostname` 自动分配 GPU (pyquda_core/pyquda_comm/__init__.py:376-426) ——用户写单进程代码即可透明跑多卡;
- **自动代码生成减维护**: Cython 绑定与插件绑定均由 `pyparser` 从 C 头自动生成 (`pyquda_core/pyquda_pyx.py:140-448`), 缓解 QUDA API 频繁变更带来的手工维护负担。

4.2 演进脉络

未验证: 本库非独立仓库, 无法从其自身 git 历史考证演进。从代码现状可推断的结构性迹象 (推断):

- **通用层下沉**: `pyquda_comm` 已是独立公共包, 且 `pyquda/field.py` 以 re-export + 扩展方式叠加高层方法 (`pyquda_core/pyquda/field.py:8-31`), 说明数据类曾经历“从 `pyquda` 抽出为 `pyquda_comm`”的重构;
- **后端扩展**: `BackendType` 含 `dpnp/torch` (`pyquda_core/pyquda_comm/array.py:6-7`), 而 README 早期版本仅强调 CuPy (README.md:5) ——后端面持续扩宽;
- **插件框架化**: `pyquda_plugins` 提供通用头文件 → Cython 插件生成器 (`pyquda_plugins/plugin_pyx.py:232-281`) `pycontract/pygwu` 只是两个已实现插件——框架先行, 插件后补。

4.3 典型 workflow

一次典型的“从 gauge 场到物理可观测量” workflow（以 `examples/3_Pion_Proton_2pt.py` 为蓝本，`examples/3_Pion_Proton_2pt.py:10-33`）：

```
1 核心init -> LatticeInfo -> getDirac(质量/精度/多重网格)
2  -> readChromaQIOGauge(gauge) -> stoutSmear(链接平滑)
3  -> dirac.loadGauge -> 循环 t_src:
4      core.invert(dirac, "wall", t_src) # 传播子反演
5      contract(传播子卷积)             # 2pt 关联函数
6  -> gatherLattice(MPI 归并) -> 拟合质量/作图
```

其中传播子反演是核心步骤，第 5 节按 15 步框架完整重现。

项目思路

项目思路核心总结：PyQUDA 的定位是“QUDA 的 Python 遥控器”——用 `pyquda_comm` 统一 MPI+ 后端 + 场数据结构，用自动生成器消灭 Cython 手写负担，让用户在 Python 里以 CuPy/Torch 张量语法完成格点 QCD 全流程，重活全部交给 QUDA。

5 主题分析：PyQUDA 传播子反演全流程

本节按 **A 代码工作框架（15 步）** 重现“从 gauge 场反演夸克传播子”这一项具体工作。被分析对象是纯 Python 代码编排 + QUDA C++ 求解的组合，判定为**代码/软件工作**，故采用 A 框架；其中涉及物理的部分（奇偶分块、 κ 、 $D+m$ 矩阵）并入相应步骤说明。

5.1 A1 目的与前期准备

为什么存在：夸克传播子 $S(x, y) = M^{-1}(x, y)$ ($M = D + m$ 为 Dirac 矩阵) 是格点 QCD 几乎所有物理量的原材料（2pt 关联函数、三点函数、核子谱等）。PyQUDA 提供一条“source $\rightarrow$ invert $\rightarrow$ setFermion”的传播子生产线（`pyquda_utils/core.py:61-82`）。

前置条件：（1）已编译 QUDA 并设 `QUDA_PATH`（`README.md:46-52`）；（2）安装 PyQUDA 与 `PyQUDA-Uutils`（`pyproject.toml:15-17`）；（3）运行时依赖 `mpi4py/numpy`（`pyquda_core/requirements.txt:1-2`）与所选后端（`cupy/torch/dpnp`）。workflow 起点是 `core.init`（`examples/2_Quark_Propagator.py:8`）。

5.2 A2 输入是什么

数据输入：规范场配置文件（Chroma QIO 格式 `.lime`），例如示例中的 `.../C24P29/..._L24x72_cfg_48000.lime`（`examples/2_Quark_Propagator.py:12`）。**参数输入：**

- 几何：`LatticeInfo([24,24,24,72], -1, 1.0)`（尺寸/反周期 t 边界/各向异性）（`examples/2_Quark_Propagator.py:10`）
- 物理参数：质量 `-0.2770`、精度 `1e-12`、迭代上限 `1000`、各向异性参数 `1.160920226`、多重网格块 `[[6,6,6,4],[4,4,4,9]]`（`examples/2_Quark_Propagator.py:11`）；
- MPI 网格：`core.init([1,1,1,2], ...)`（2 卡沿 t 切分）（`examples/2_Quark_Propagator.py:8`）。

依赖输入：QUDA 库、Cython 生成物（`quda.pyx` 由构建期生成，`pyquda_core/setup.py:8-10`）。

5.3 A3 输出应该是什么

预期产物：夸克传播子 S (LatticePropagator, 形状 (2,Lt,Lz,Ly,Lx/2,4,4,3,3) 奇偶分块) 与派生的 π 介子 2pt 关联函数 (examples/2_Quark_Propagator.py:19-27), 最终输出 pion_2pt.svg 对数坐标图 (examples/2_Quark_Propagator.py:30-39)。物理上 $2pt \propto e^{-m_\pi t}$, 斜率给出 π 质量。

5.4 A4 整体框架

主流程分四段：

```

1 [0 初始化] core.init(grid, latt)          # pyquda_core/pyquda/__init__.py:99
2 [1 配置]   LatticeInfo + getDirac(...)    # pyquda_utils/core.py:388
3 [2 输入]   readQIOGauge + stoutSmear      # pyquda_io/chroma.py:10, field.py:224
4 [3 反演]   循环 t_src: wall源 -> invert -> setFermion -> contract
5           # pyquda_utils/core.py:61 / dirac/abstract.py:212
6 [4 分析]   gatherLattice -> roll -> mean -> plot
7           # pyquda_utils/core.py:315

```

模块归属：步骤 0/1 在 pyquda + pyquda_utils, 2 在 pyquda_io + field, 3 横跨 utils/core → dirac → src/quda.in.pyx, 4 回到 utils/core。

5.5 A5 关键细节

关键函数：

- `core.invert(dirac, "wall", t_src)`: 按 $N_s \times N_c = 12$ 个源分量分批 (mrhs 路多重源), 每次 `source.source` 构造源、`invertMultiSrcRestart` 反演、`propag.setFermion` 组装 (pyquda_utils/core.py:61-82);
- `FermionDirac.invertMultiSrc`: 设 `num_src` 后调 `invertMultiSrcQuda` (pyquda_core/pyquda/dirac/abstract.py:252-255); 底层是 Cython 直绑 (pyquda_core/pyquda/src/quda.in.pyx:252-255);
- 多重网格: Multigrid 实例持有 `QudaMultigridParam`, 首次加载 gauge 时 `new()` 建 MG 预条件器, 后续 `update()` 仅薄更新 (pyquda_core/pyquda/dirac/general.py:478-543)。

易错点：`kappa` 由质量换算 (pyquda_core/pyquda/dirac/wilson.py:19); 反周期 `t` 边界与各向异性需在 `loadGauge` 前对 gauge 副本生效 (pyquda_core/pyquda/dirac/general.py:568-580); 源位置需按 `gt*Lt <= t < (gt+1)*Lt` 判断本地归属 (pyquda_utils/source.py:55-66)。

5.6 A6 任务如何正确执行

标准执行路径 (正确做法):

```

1 # 1) 初始化 (直接 Python API)
2 python -c "from pyquda_utils import core; core.init([1,1,1,2], ...)"
3 # 2) 或经 CLI 包装 (__main__.py 解析后 exec 脚本)
4 python -m pyquda -g 1 1 1 2 -l 24 24 24 72 script.py
5 # 3) 反演核心调用
6 dirac = core.getDirac(latt_info, mass, tol, maxiter, multigrid=[[6,6,6,4],[4,4,4,9]])

```



```

7 with dirac.useGauge(gauge):
8     propag = core.invert(dirac, "wall", t_src) # 退出上下文自动 freeGauge

```

useGauge 上下文管理器保证 gauge 的 load/free 配对, 栈式管理多 gauge (pyquda_core/pyquda/dirac/abstract.py:5)

5.7 A7 实际执行情况

未验证: 本环境无 QUDA 库与 GPU, 无法实跑反演。作为替代, 仓库自带 tests/ 以 weak_field.lime 弱场配置验证同路径(tests/check_pyquda.py:1-31, tests/test_clover.py:7-17): core.init(None, [4,4,4,8]) → getClover → useGauge → core.invert(..., "point", [0,0,0,0]), 并同 Chroma 参考传播子逐位比对 (tests/test_clover.py:16-21)。该测试路径与本节 15 步框架的步骤 0-4 完全一致, **推断**其可作为本工作流的可行等价物。

5.8 A8 实际输出情况

未验证: 未实跑故无实测产物。仓库测试的输出形态可从代码推知: 校验量 (propagator - propagator_chroma).norm2()*0.5 (残差范数, tests/test_clover.py:21) ; 示例输出为 pion_2pt.svg 与对数坐标质量图(examples/2_Quark_Propagator.py:35-37, examples/3_Pion_Proton_2pt.py:76-)

5.9 A9 结果分析

未验证: 无本机实测图表。仓库的自校验逻辑提供了“结果合理”的判定标准: (1) 传播子与 Chroma 参考逐位一致(tests/test_clover.py:21); (2) 2pt 对数图应为直线(examples/2_Quark_Propagator.py:35-37); (3) 有效质量图在平台区取值稳定 (examples/3_Pion_Proton_2pt.py:80-83)。此外 test_io.py 验证读写往返 (IO 层), test_pycontract.py 对比 opt_einsum 与插件收缩 (tests/test_io.py:1-47, tests/test_pycontract.py:1-380) ——这些构成了“全链正确性”的实测证据面。

5.10 A10 综合分析

代码-对象映射: 本工作流中每个代码步骤对应一个物理动作:

- source.wall ↔ 在 $t = t_{src}$ 平面置 $\eta = 1$ (夸克源) (pyquda_utils/source.py:55-66);
- dirac.invert(b) ↔ 求解 $M\psi = \eta$, 即 $\psi = S\eta$ (pyquda_core/pyquda/dirac/abstract.py:212-216);
- propag.setFermion(x, spin, color) ↔ 把 12 个源分量解拼成 4×3 矩阵传播子 (pyquda_core/pyquda_comm/f);
- contract("wtzyxjiba,wtzyxjiba->t", ...) ↔ $\sum_{xyz,\alpha\beta,a} |S(x,t)|^2$ 求 π 2pt (examples/2_Quark_Propagator.);
- gatherLattice ↔ 多卡子格点沿 t 拼接为整格点 2pt (pyquda_utils/core.py:315-385)。

深层原因: 反演之所以要“12 个源分量 + 多重源批量”, 是因为 S 是矩阵 M^{-1} , 其列由 12 个标准基源张成 ($N_s \times N_c = 4 \times 3 = 12$); 批量求解由 QUDA invertMultiSrcQuda 一次调用完成多右端项 (pyquda_core/pyquda/src/quda.in.pyx:252-255), 大幅摊薄开销。

5.11 A11 结尾总结

结论

结论：PyQUDA 的传播子反演是一条**薄封装直链**——`utils/core.invert` 编排 12 源分量与多重源批量，`dirac` 层做参数工厂与上下文管理，`quda.in.pyx` 零拷贝直调 QUDA 求解器；用户侧只需一行 `core.invert(dirac, "wall", t_src)` 即得到物理传播子。整条链在仓库测试中以 Chroma 参考验证。

5.12 A12 结尾评价

优点：接口极简（一行反演）、后端可插拔、MPI 透明、构建期自动生成 Cython 绑定。**可改进处（推断）：**反演参数（精度/迭代上限/MG 块）需用户显式指定，缺少面向结果的自动容差策略；示例中 `t_src` 全时间面逐点循环（`examples/2_Quark_Propagator.py:16`）在资源上可进一步合并。

5.13 A13 补充说明

未覆盖项：A7/A8/A9 因无 QUDA/GPU 环境而**未实跑**，仅以仓库自测与代码逻辑佐证；`examples/1_Quenched_HMC.ipynb`（1598 行 notebook）仅索引未精读；vendored `pyparser`（126 文件）与 QUDA 头文件（137 个 h）仅索引。插件 `pygw` 的 overlap 反演链未展开（非传播子反演主线）。

5.14 A14 参考源

本步证据汇总见第 7 节参考源清单；与 A1–A13 对应的核心证据包括：`pyquda_utils/core.py:61–82`、`pyquda_core/pyquda/dirac/abstract.py:212–260`、`pyquda_core/pyquda/dirac/general.py:400–475`、`pyquda_core/pyquda/src/quda.in.pyx:247–260`、`pyquda_utils/source.py:25–66`、`examples/2_Quark_Propagator.py:1–39`。

5.15 A15 附录与附件

附： workflow 关键代码直引见第 6 节片段 1（`core.invert`）、片段 2（`invert/ invertMultiSrc`）、片段 3（示例脚本全文）、片段 4（QIO 读取）。本报告会话日志为工作目录 `.analy.< 时间戳 >.log`（不入库）。

6 关键代码/文档片段

以下代码一律以

```
1 %%
2 %% This is file `.tex',
3 %% generated with the docstrip utility.
4 %%
5 %% The original source files were:
6 %%
```

```

7 %% fileerr.dtx (with options: `return')
8 %%
9 %% This is a generated file.
10 %%
11 %% The source is maintained by the LaTeX Project team and bug
12 %% reports for it can be opened at https://latex-project.org/bugs/
13 %% (but please observe conditions on bug reports sent to that address!)
14 %%
15 %%
16 %% Copyright (C) 1993-2021
17 %% The LaTeX Project and any individual authors listed elsewhere
18 %% in this file.
19 %%
20 %% This file was generated from file(s) of the Standard LaTeX `Tools Bundle'.
21 %% -----
22 %%
23 %% It may be distributed and/or modified under the
24 %% conditions of the LaTeX Project Public License, either version 1.3c
25 %% of this license or (at your option) any later version.
26 %% The latest version of this license is in
27 %% https://www.latex-project.org/lppl.txt
28 %% and version 1.3c or later is part of all distributions of LaTeX
29 %% version 2005/12/01 or later.
30 %%
31 %% This file may only be distributed together with a copy of the LaTeX
32 %% `Tools Bundle'. You may however distribute the LaTeX `Tools Bundle'
33 %% without such generated files.
34 %%
35 %% The list of all files belonging to the LaTeX `Tools Bundle' is
36 %% given in the file `manifest.txt'.
37 %%
38 \message{File ignored}
39 \endinput
40 %%
41 %% End of file `.tex'.

```

直引仓库原文件（每段 ≤ 40 行）。

片段 1: 传播子反演编排 (pyquda_utils/core.py:61-82)

```

1 def invert(
2     dirac: FermionDirac,
3     source_type: Literal["point", "wall", "volume", "momentum", "colorvector"],
4     t_srce: Union[List[int], int, None],
5     source_phase=None,
6     mrhs: int = 1,
7     restart: int = 0,
8 ):
9     latt_info = dirac.latt_info
10
11     propag = LatticePropagator(latt_info)
12     s = 0
13     while s < Ns * Nc:
14         b = MultiLatticeFermion(latt_info, min(mrhs, Ns * Nc - s))
15         for i in range(b.L5):
16             b[i] = source.source(latt_info, source_type, t_srce, (s + i) // Nc, (s + i) % Nc, source_phase)

```

```

17     x = dirac.invertMultiSrcRestart(b, restart)
18     for i in range(b.L5):
19         propag.setFermion(x[i], (s + i) // Nc, (s + i) % Nc)
20     s += mrhs
21
22     return propag

```

片段 2: Dirac 反演入口与多重源 (dirac/abstract.py:212-224,247-260)

```

1     def invert(self, b: LatticeFermion):
2         x = LatticeFermion(b.latt_info)
3         invertQuda(x.data_ptr, b.data_ptr, self.invert_param)
4         self.performance()
5         return x
6
7     def invertRestart(self, b: LatticeFermion, restart: int):
8         x = self.invert(b)
9         if restart > 0:
10             self.invert_param.solver_normalization = QudaSolverNormalization.QUDA_SOURCE_NORMALIZATION
11             x += self.invertRestart(b - self.mat(x), restart - 1)
12             self.invert_param.solver_normalization = QudaSolverNormalization.QUDA_DEFAULT_NORMALIZATION
13         return x

```

```

1     def invertMultiSrc(self, b: MultiLatticeFermion):
2         self.invert_param.num_src = b.L5
3         x = MultiLatticeFermion(b.latt_info, b.L5)
4         invertMultiSrcQuda(x.data_ptrs, b.data_ptrs, self.invert_param)
5         self.performance()
6         return x
7
8     def invertMultiSrcRestart(self, b: MultiLatticeFermion, restart: int):
9         x = self.invertMultiSrc(b)
10        if restart > 0:
11            self.invert_param.solver_normalization = QudaSolverNormalization.QUDA_SOURCE_NORMALIZATION
12            x += self.invertMultiSrcRestart(b - self.matMultiSrc(x), restart - 1)
13            self.invert_param.solver_normalization = QudaSolverNormalization.QUDA_DEFAULT_NORMALIZATION
14        return x

```

片段 3: 端到端示例 (examples/2_Quark_Propagator.py 全文 39 行)

```

1     import numpy as np
2     import cupy as cp
3     from opt_einsum import contract
4     from matplotlib import pyplot as plt
5
6     from pyquda_utils import core, io, gamma, source
7
8     core.init([1, 1, 1, 2], resource_path=".cache/quda")
9     latt_info = core.LatticeInfo([24, 24, 24, 72], -1, 1.0)
10
11     dirac = core.getDirac(latt_info, -0.2770, 1e-12, 1000, 1.0, 1.160920226, 1.160920226, [[6, 6, 6, 4], [4, 4, 4, 9]])
12     gauge = io.readChromaQIOGauge("/public/ensemble/C24P29/beta6.20_mu-0.2770_ms-0.2400_L24x72_cfg_48000.lime")
13     gauge.stoutSmear(1, 0.125, 4)
14
15     G5 = gamma.gamma(15)
16     t_src_list = list(range(0, 72, 1))

```

```

17 pion = cp.zeros((len(t_src_list), latt_info.Lt), "<f8")
18
19 propag = core.LatticePropagator(latt_info)
20 dirac.loadGauge(gauge)
21 for t_idx, t_src in enumerate(t_src_list):
22     for spin in range(latt_info.Ns):
23         for color in range(latt_info.Nc):
24             b = source.wall(latt_info, t_src, spin, color)
25             x = dirac.invert(b)
26             propag.setFermion(x, spin, color)
27     pion[t_idx] += contract("wtzyxjiba,wtzyxjiba->t", propag.data.conj(), propag.data).real
28 dirac.freeGauge()
29
30 tmp = core.gatherLattice(pion.get(), [1, -1, -1, -1])
31 if latt_info.mpi_rank == 0:
32     for t_idx, t_src in enumerate(t_src_list):
33         tmp[t_idx] = np.roll(tmp[t_idx], -t_src, 0)
34     twopt = tmp.mean(0)
35     plt.plot(np.arange(72), twopt, "-")
36     plt.yscale("log")
37     plt.savefig("pion_2pt.svg")
38     # np.save("pion.npy", tmp)
39     print(tmp)

```

片段 4: Chroma QIO 规范场读取 (pyquda_io/chroma.py:10-38)

```

1 def readQIOGauge(filename: str, checksum: bool = True, reunitarize_sigma: float = 1e-6):
2     from .lime import Lime
3
4     filename = path.expanduser(path.expandvars(filename))
5     lime = Lime(filename)
6     scidac_private_file_xml = lime.loadXML("scidac-private-file-xml")
7     scidac_private_record_xml = lime.loadXML("scidac-private-record-xml")
8     scidac_checksum_xml = lime.loadXML("scidac-checksum")
9     offset = lime.record("ildg-binary-data").offset
10
11     precision = _precision_map[scidac_private_record_xml.find("precision").text]
12     assert int(scidac_private_record_xml.find("colors").text) == Nc
13     assert int(scidac_private_record_xml.find("typesize").text) == Nc * Nc * 2 * precision
14     assert int(scidac_private_record_xml.find("datacount").text) == Nd
15     assert int(scidac_private_file_xml.find("spacetime").text) == Nd
16     latt_size = [int(L) for L in scidac_private_file_xml.find("dims").text.split()]
17     Lx, Ly, Lz, Lt = getSublatticeSize(latt_size)
18     dtype = f">c{2 * precision}"
19
20     gauge = readMPIFile(filename, dtype, offset, (Lt, Lz, Ly, Lx, Nd, Nc, Nc), (3, 2, 1, 0))
21     if checksum:
22         assert (
23             int(scidac_checksum_xml.find("suma").text, 16),
24             int(scidac_checksum_xml.find("sumb").text, 16),
25         ) == checksumSciDAC(latt_size, gauge.reshape(Lt * Lz * Ly * Lx, Nd * Nc * Nc))
26     gauge = gauge.transpose(4, 0, 1, 2, 3, 5, 6).astype("<c16")
27     if precision == 4:
28         gauge = gaugeReunitarize(gauge, reunitarize_sigma)
29     return latt_size, gauge

```

片段 5: QUDA 初始化与网格 (pyquda/___init___py:60-96)

```

1 def initQUDA(grid_size: List[int], device: int, use_malloc_quda: bool = False):
2     import atexit
3     from . import quda
4
5     global _QUDA_INITIALIZED
6     if not isGridInitialized() or not isDeviceInitialized():
7         getLogger().critical("initGrid and initDevice should be called before initQUDA", RuntimeError)
8     backend = getArrayBackend()
9     backend_target = getArrayBackendTarget()
10
11     if use_malloc_quda:
12         import ctypes
13         import sysconfig
14
15         malloc_quda = ctypes.CDLL(
16             path.join(path.dirname(path.abspath(__file__)), "malloc_quda" + sysconfig.get_config_var("EXT_SUFFIX"))
17         )
18         quda_malloc = ctypes.cast(getattr(malloc_quda, f"{backend}_malloc"), ctypes.c_void_p).value
19         quda_free = ctypes.cast(getattr(malloc_quda, f"{backend}_free"), ctypes.c_void_p).value
20
21         if backend == "cupy" and backend_target in ("cuda", "hip"):
22             import cupy
23
24             assert quda_malloc is not None and quda_free is not None
25             allocator = cupy.cuda.CFunctionAllocator(0, quda_malloc, quda_free, None).malloc
26             cupy.cuda.set_allocator(allocator)
27         elif backend == "torch" and backend_target in ("cuda", "hip"):
28             import torch
29
30             assert quda_malloc is not None and quda_free is not None
31             allocator = torch._C._cuda_customAllocator(quda_malloc, quda_free)
32             torch._C._cuda_changeCurrentAllocator(allocator)
33
34     quda.initCommsGridQuda(4, grid_size, getGridRanks())
35     quda.initQuda(device if backend_target != "cpu" else -1)
36     atexit.register(quda.endQuda)
37     _QUDA_INITIALIZED = True

```

片段 6: Cython 直绑 invertQuda (src/quda.in.pyx:247-255)

```

1 def invertQuda(h_x, h_b, QudaInvertParam param):
2     _h_x = _NDArray(h_x, 1)
3     _h_b = _NDArray(h_b, 1)
4     quda.invertQuda(_h_x.ptr, _h_b.ptr, &param.param)
5
6 def invertMultiSrcQuda(_hp_x, _hp_b, QudaInvertParam param):
7     __hp_x = _NDArray(_hp_x, 2)
8     __hp_b = _NDArray(_hp_b, 2)
9     quda.invertMultiSrcQuda(__hp_x.ptrs, __hp_b.ptrs, &param.param)

```

片段 7: Cython 直绑 initCommsGridQuda (src/quda.in.pyx:187-196)

```

1 def initCommsGridQuda(int nDim, list dims, list ranks):
2     map_data.ndim = nDim
3     map_data.size = 1

```

```

4   for i in range(map_data.ndim):
5       map_data.dims[i] = dims[i]
6       map_data.size *= dims[i]
7   map_data.ranks = <int *>malloc(map_data.size * sizeof(int))
8   for i in range(map_data.size):
9       map_data.ranks[i] = ranks[i]
10  quda.initCommsGridQuda(map_data.ndim, map_data.dims, commsMap, <void *>(&map_data))

```

片段 8: getDefaultGrid 通信量最小化 (pyquda_comm/___init___py:222-262)

```

1  def getDefaultGrid(mpi_size: int, shared_size: int, latt_size: Sequence[int], evenodd: bool = True):
2      Lx, Ly, Lz, Lt = latt_size
3      latt_vol = Lx * Ly * Lz * Lt
4      assert latt_vol % mpi_size == 0, "lattice volume must be divisible by MPI size"
5      node_size = mpi_size // shared_size
6      node_latt_vol = latt_vol // node_size
7      local_latt_vol = latt_vol // mpi_size
8      min_comm, min_grid = (4 * 2 * node_latt_vol, 4 * 2 * local_latt_vol, 4 * 2), []
9      if evenodd:
10         assert (
11             Lx % 2 == 0 and Ly % 2 == 0 and Lz % 2 == 0 and Lt % 2 == 0
12         ), "lattice size must be even in all directions for even-odd preconditioning"
13         Lx_, Ly_, Lz_, Lt_ = Lx // 2, Ly // 2, Lz // 2, Lt // 2
14     else:
15         Lx_, Ly_, Lz_, Lt_ = Lx, Ly, Lz, Lt
16     node_partition = _partition(node_size, [Lx_, Ly_, Lz_, Lt_])
17     for node_grid_size in node_partition:
18         Nx, Ny, Nz, Nt = node_grid_size
19         shared_partition = _partition(shared_size, [Lx_ // Nx, Ly_ // Ny, Lz_ // Nz, Lt_ // Nt])
20         for shared_grid_size in shared_partition:
21             grid_size = [node_grid_size[dir] * shared_grid_size[dir] for dir in range(4)]
22             comm, nic_size, nic_count = 0, 0, 0
23             for dir in range(4):
24                 if node_grid_size[dir] > 1:
25                     node_latt_surf = node_latt_vol // (latt_size[dir] // node_grid_size[dir])
26                     local_latt_surf = local_latt_vol // (latt_size[dir] // grid_size[dir])
27                     comm += 2 * node_latt_surf
28                     if shared_grid_size[dir] == 1:
29                         nic_size += 2 * local_latt_surf
30                         nic_count += 2
31                     else: # shared_grid_size[dir] > 1
32                         nic_size += 1 * local_latt_surf
33                         nic_count += 1
34
35             if (comm, nic_size, nic_count) < min_comm:
36                 min_comm, min_grid = (comm, nic_size, nic_count), [(grid_size, shared_grid_size)]
37             elif (comm, nic_size, nic_count) == min_comm:
38                 min_grid.append((grid_size, shared_grid_size))
39     if min_grid == []:
40         raise ValueError(f"Cannot get proper grid for lattice size {latt_size} with {mpi_size} MPI processes")
41     return min(min_grid)

```

片段 9: PYQUDA 后端设备抽象 (pyquda_comm/array.py:19-40)

```

1  def backendDeviceAPI(backend: BackendType, backend_target: BackendTargetType):
2      if backend == "numpy":

```